

EduVision School Website Platform

A production-ready, multi-tenant School Website Content Management System. Every school gets its own public website, secure admin dashboard, branding, and isolated data. A super admin dashboard provisions new schools and their admin accounts automatically.

Built for commercial deployment: VPS, dedicated server, or any cloud provider. See `DEPLOYMENT.md` and `docs/deployment/` for Ubuntu, DigitalOcean, Hetzner, AWS and Azure guides.

Latest release: v1.3.0 (desktop-public branch) — the public school website is now packaged as a standalone desktop application using Electron. It launches from a desktop icon, starts its own local API and web server, and opens in its own window. EduVision-Web continues to contain both the public website and the School CMS; the desktop build disables CMS routes so the app is a public-only launcher. See `RELEASE_NOTES_v1.3.0.md` and `CHANGELOG.md`.

Desktop Application

The `apps/desktop` Electron shell wraps the public Next.js website and a local NestJS API. It is designed for schools that want to run their public website locally on Ubuntu or Windows and optionally publish it later.

Run from source (Ubuntu)

```
# Start PostgreSQL and Redis first
docker compose up -d postgres redis
```

```
# Build the API and web app
npm run build --workspace=@eduvision/api
npm run build --workspace=web
```

```
# Launch the desktop app
npm run desktop:start
```

Install a desktop entry (Ubuntu)

```
npm run desktop:install
```

This creates `~/.local/share/applications/eduvision-school-website.desktop` so the app appears in the applications menu.

Windows

Use `scripts/desktop-start.bat` from a command prompt after installing Node.js and running `npm install`.

How it works

- `apps/desktop/src/main.js` starts the API (`apps/api/dist/src/main.js`) on port 4000 and the Next.js standalone server (`apps/web/.next/standalone/apps/web/server.js`) on port 3000.
- It then opens a 1280x800 Electron window pointing at `http://localhost:3000`.
- Admin routes (`/admin`, `/setup`, etc.) are redirected back to `/` by `apps/web/src/middleware.ts` when `DISABLE_ADMIN=true`, keeping the desktop build public-only.
- The CMS is still part of EduVision-Web and is accessible through the normal web deployment at `https://yourdomain.com/admin`.
- The default school tenant is `demo-school`; set `SCHOOL_SLUG` to switch schools.

Features

- Public school website with dynamic CMS pages: Home, About, Principal's Message, Vision & Mission, History, Staff, Academics, Admissions, School Fees, Uniform, Sports, Gallery, News, Events, Downloads, Contact, Emergency Contacts, Social Media.
- Secure admin dashboard with full CRUD for pages, posts, events, staff, galleries, downloads, fees, contacts, social links, navigation, users, students, notices, admissions, results, attendance, subscriptions, invoices and licenses.
- Media library for logos, photos, videos, PDFs, newsletters, and policies with local or S3-compatible storage.
- School branding: colors, logo, favicon, dark mode, custom domain support.
- Super admin dashboard with platform statistics, subscriptions, licences, invoices, audit logs, one-click school creation, system health, storage and backup reporting.
- Parent, Teacher and Learner portals with notices, calendar, results and attendance.
- Subscription, billing, trial and licence management with expired-account enforcement.
- Two-factor authentication (TOTP) and password-reset via email.
- Redis-backed caching for public endpoints.
- Automated daily database backups, production error logging and health monitoring.
- Multi-tenant isolation: each school has its own data, users, branding, and domain.
- JWT authentication, bcrypt password hashing, role-based access control, audit logs, rate limiting, CSP, and security headers.
- Mobile-responsive, accessible UI with dark and light mode.

Technology Stack

- **Frontend:** Next.js 15 + React 19 + Tailwind CSS 4 + Radix UI components
- **Backend:** NestJS 11 + Prisma 5 + PostgreSQL 16
- **Reverse Proxy / SSL:** Nginx + Certbot (Let's Encrypt)
- **Containerization:** Docker Compose
- **Authentication:** JWT, bcrypt, Passport, httpOnly cookies
- **Security:** Helmet, CORS, rate limiting, CSP, XSS/CSRF protection, audit logs, RBAC

Quick Start (Local Development)

Prerequisites

- Node.js 20
- Docker and Docker Compose
- npm

1. Install Dependencies

```
npm install
```

2. Configure Environment

```
cp .env.example .env
```

The default `.env.example` is configured for local development (`API_URL=http://localhost:4000`, `STORAGE_BASE_URL=http://localhost:4000`).

3. Start PostgreSQL and Redis

```
docker compose up -d postgres redis
```

4. Run Migrations and Seed the Database

```
cd apps/api
npx prisma migrate dev
npx tsx prisma/seed.ts
cd ../../
```

5. Start the Development Servers

In separate terminals:

```
# Terminal 1 - API
cd apps/api && npm run dev    # http://localhost:4000
```

```
# Terminal 2 - Web
cd apps/web && npm run dev    # http://localhost:3000
```

6. Log In

- **Super admin:** superadmin@eduvision.local / SuperAdmin123! (school slug: platform)
- **Demo school admin:** admin@demo-school.edu / SchoolAdmin123! (school slug: demo-school)

Change all default credentials immediately after first login.

Production Deployment

For the complete production deployment guide, see [DEPLOYMENT.md](#).

The short version:

```
cp .env.example .env
# edit .env with real secrets and domain
./scripts/deploy.sh
```

Then apply real SSL:

```
./scripts/init-ssl.sh yourdomain.com www.yourdomain.com
```

Environment Variables

Copy `.env.example` to `.env` and configure at minimum:

Variable	Purpose
POSTGRES_PASSWORD	PostgreSQL password
JWT_SECRET	Long random string for JWT signing
COOKIE_DOMAIN	Root domain with leading dot, e.g. .eduvisionschools.co.za
STORAGE_BASE_URL	Public HTTPS domain for uploaded files
API_URL	Public API base URL

See [DEPLOYMENT.md](#) for the full environment reference.

Project Structure

```
eduvision/
├── apps/
│   ├── api/                # NestJS backend
│   │   ├── src/
│   │   ├── prisma/
│   │   └── Dockerfile
│   └── web/                # Next.js frontend
│       ├── src/
│       ├── public/
│       └── Dockerfile
├── nginx/
│   ├── default.conf        # Nginx reverse proxy config
│   └── ssl/                # SSL certificates
├── scripts/
│   ├── deploy.sh           # One-command production deploy
│   ├── init-selfsigned.sh  # Generate self-signed cert for first boot
│   └── init-ssl.sh          # Obtain Let's Encrypt certificate
├── docker-compose.yml       # Development stack
├── docker-compose.prod.yml  # Production stack
├── .env.example             # Environment template
├── README.md               # This file
├── DEPLOYMENT.md           # Production deployment guide
└── RELEASE_NOTES_v1.0.0.md # Release notes
```

Type Checking and Building

```
# API
cd apps/api && npx tsc --noEmit

# Web
cd apps/web && npx tsc --noEmit

# Production web build
cd apps/web && npm run build
```

Backups

See [DEPLOYMENT.md](#) for automated database and file backup instructions.

Monitoring and Scaling

The platform is stateless except for PostgreSQL, Redis, and the uploads volume. To scale horizontally, move the database and cache to managed services and use S3 for file storage. Details in [DEPLOYMENT.md](#).

Future Modules

The architecture supports adding: Parent Portal, Teacher Portal, Learner Portal, Online Admissions, Results, Attendance, Timetable, AI Chatbot, SMS/Email Notifications, Online Payments, Library Management, Hostel Management, and Transport Management.

License

Copyright © 2026 EduVision. All rights reserved.